Information Processing Letters 173 (2022) 106174

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Palindromic trees for a sliding window and its applications

Takuya Mieno [a,b,*], Kiichi Watanabe [a], Yuto Nakashima [a], Shunsuke Inenaga [a,c], Hideo Bannai [d], Masayuki Takeda [a]

[a] Department of Informatics, Kyushu University, Japan
[b] Japan Society for the Promotion of Science, Japan
[c] PRESTO, Japan Science and Technology Agency, Japan
[d] M&D Data Science Center, Tokyo Medical and Dental University, Japan

ARTICLE INFO

Article history:
Received 15 April 2021
Received in revised form 10 June 2021
Accepted 22 July 2021
Available online 8 August 2021
Communicated by Ranko Lazic

Keywords:
String algorithms
Data structures
Palindromes
Sliding window

ABSTRACT

The palindromic tree (a.k.a. eertree) for a string S of length n is a tree-like data structure that represents the set of all distinct palindromic substrings of S, using $O(n)$ space [Rubinchik and Shur, 2018]. It is known that, when S is over an alphabet of size σ and is given in an online manner, then the palindromic tree of S can be constructed in $O(n \log \sigma)$ time with $O(n)$ space. In this paper, we consider the sliding window version of the problem: For a sliding window of length at most d, we present two versions of an algorithm which maintains the palindromic tree of size $O(d)$ for every sliding window $S[i..j]$ over S, where $1 \leq j - i + 1 \leq d$. The first version works in $O(n \log \sigma')$ time with $O(d)$ space where $\sigma' \leq d$ is the maximum number of distinct characters in the windows, and the second one works in $O(n + d\sigma)$ time with $(d+2)\sigma + O(d)$ space. We also show how our algorithms can be applied to efficient computation of minimal unique palindromic substrings (MUPS) and minimal absent palindromic words (MAPW) for a sliding window.

1. Introduction

Palindromes. A *palindrome* is a string that reads the same forward and backward. Palindromic structures in strings have been heavily studied in the fields of string processing algorithms and combinatorics on strings [1–6]. One of the most famous results related to palindromic structures is Manacher's algorithm [1], which computes all maximal palindromes in a given string S. Manacher's algorithm essentially computes all palindromes in S since any palindromic substring of S is a substring of some maximal palindrome in S. Another interesting topic is to enumerate distinct palindromes in a string. It is known that any string of length n contains at most $n + 1$ distinct palindromes, including the empty string [7]. Groult et al. [2] proposed an $O(n)$-time algorithm which enumerates all distinct palindromes in a given string of length n over an integer alphabet of size $\sigma = n^{O(1)}$. For the same problem in the *online* model, Kosolobov et al. [3] proposed an $O(n \log \sigma)$-time and $O(n)$-space algorithm for a general ordered alphabet. Kosolobov et al.'s algorithm is a combination of Manacher's algorithm and Ukkonen's online suffix tree construction algorithm [8]. Rubinchik and Shur [4] proposed a new data structure called *eertree*, which permits efficient access to distinct palindromes in a string *without* storing the string itself. Eertrees can be utilized for solving problems related to palindromic structures, such as the palindrome counting problem and the palindromic factorization problem [4]. The size of the eertree of S is linear in the number p_S of distinct palindromes in S [4]. Since p_S is at most $|S| + 1$, the size of the eertree of S can be

* Corresponding author at: Department of Informatics, Kyushu University, Japan.
E-mail addresses: takuya.mieno@inf.kyushu-u.ac.jp (T. Mieno), yuto.nakashima@inf.kyushu-u.ac.jp (Y. Nakashima), inenaga@inf.kyushu-u.ac.jp (S. Inenaga), hdbn.dsc@tmd.ac.jp (H. Bannai), takeda@inf.kyushu-u.ac.jp (M. Takeda).

https://doi.org/10.1016/j.ipl.2021.106174

much smaller than the length $|S|$ of the string, e.g., for $S = (\texttt{abc})^{n/3}$, $p_S = 4$ since all distinct palindromes in S are a, b, c, and the empty string. Thus, the size of the eertree of S can be much smaller than that of the suffix tree of S, which is $\Theta(n)$. Therefore, it is of significance if one can build eertrees without suffix trees. Rubinchik and Shur [4] indeed proposed an online eertree construction algorithm running in $O(n \log \sigma)$ time without suffix trees.

Recently, a concept of palindromic structures called *minimal unique palindromic substrings* (*MUPS*) is introduced by Inoue et al. [9]. A palindromic substring $w = S[i..j]$ of a string S is called a MUPS of S if w occurs in S exactly once, and $S[i+1..j-1]$ occurs at least twice in S. MUPSs are utilized for solving the *shortest unique palindromic substring* (*SUPS*) problem [9], which is motivated by an application in molecular biology. Watanabe et al. [10] proposed an algorithm to solve the SUPS problem based on the *run-length encoding* (*RLE*) version of eertrees, named $\texttt{e}^2\texttt{rtre}^2$.

Sliding window model. In this paper, we consider the problems of computing palindromic structures for the *sliding window* model. The sliding window model is a natural generalization of the online model, and the assumptions of this model are natural when we need to process a massive or a streaming string data with limited memory space. A typical and classical application to the sliding window model is data compression, such as Lempel-Ziv 77 (the original version) [11] and PPM [12]. Note that sliding-window Lempel-Ziv 77 is an immediate application of suffix trees for a sliding window, which can be maintained in $O(n \log \sigma')$ time using $O(d)$ space [13–15] where d is the size of the window and $\sigma' \leq d$ is the maximum number of distinct characters in every window. Recently, several algorithms for computing substrings for a sliding window with certain interesting properties are proposed: For instance, Crochemore et al. [16] introduced the problem of computing *minimal absent words* (*MAWs*) for a sliding window, and proposed an $O(n\sigma)$-time and $O(d\sigma)$-space algorithm using suffix trees for a sliding window. Mieno et al. [17] proposed an algorithm for computing *minimal unique substrings* (*MUSs*) [18] for a sliding window, in $O(n \log \sigma')$-time and $O(d)$-space, again based on suffix trees for a sliding window.

Our contributions. In this paper, we consider the problem of maintaining eertrees for the sliding window model, that is, given a string S of length n and a *window* of a fixed size d, we maintain eertrees of substrings $S[i..i+d-1]$ for incremental $i = 0, 1, \ldots, n-d$. The naïve solution that computes the eertree from scratch for every substring $S[i..i+d-1]$ requires;

- $O(n^2)$ time and $O(n)$ space assuming that the alphabets for all the sliding windows are integers of size $n^{O(1)}$;
- $O(dn)$ time and $O(d)$ space assuming that each sliding window is over an integer alphabet of size $d^{O(1)}$ or a constant-size alphabet;
- $O(dn \log \sigma')$ time and $O(d)$ space for general ordered alphabets.

We propose the first efficient algorithm which maintains eertrees for a sliding window in a total of $O(n \log \sigma')$ time using $O(d)$ space. We then give an alternative eertree construction algorithm for a sliding window which runs in $O(n + d\sigma)$ time with $(d+2)\sigma + O(d)$ space. We remark that in the case of constant-size alphabets with $\sigma = O(1)$, our solutions are the *first linear-time algorithms* for maintaining the eertrees for a sliding window.

We note that this paper is basically a theoretical one, while palindromic structures have many applications on the practical side, such as bioinformatics and molecular biology [19–22]. On the theoretical side, our methods are the *first efficient algorithms* which compute all distinct palindromes in each substring of length d in the input string, since the eertree of a string represents all distinct palindromes in the string. This gives us the first efficient methods to check the *palindromic richness* for all the substrings of fixed length d, where a string of length m is called *rich* if it contains $m+1$ distinct palindromes [23]. It has been shown in [7] and the references therein, that palindromic richness has strong connections to *Sturmian words*, a very important and widely-studied class of strings with a number of interesting combinatorial properties (cf. [24]). Thus, our algorithms can serve as useful tools for further analyses of palindromic structures of strings.

In addition, we introduce a new concept of palindromic structures called *minimal absent palindromic words* (*MAPW*) and consider the problem of maintaining MAPWs for a sliding window. A string w is called a MAPW of string S if w is a palindrome, w does not occur in S, and $w[1..|w|-2]$ occurs in S. MAPWs can be seen as a palindromic version of the notion of MAWs, which are extensively studied in the fields of string processing and bioinformatics [25–29]. As applications to our eertree algorithms, we propose an algorithm that maintains MUPSs for a sliding window in a total of $O(n \log \sigma')$ time using $O(d)$ space, and an algorithm that maintains MAPWs for a sliding window in a total of $O(n + d\sigma)$ time using $O(d\sigma)$ space.

We emphasize that our algorithms are stand-alone in the sense that they do not use suffix trees, while the majority of existing efficient sliding window algorithms (see above) make heavy use of suffix trees.

2. Preliminaries

2.1. Strings

Let Σ be an *alphabet* of size σ. An element of Σ is called a *character*. An element of Σ^* is called a *string*. The length of a string S is denoted by $|S|$. The *empty string* ε is the string of length 0. If $S = xyz$, then x, y, and z are called a *prefix*, *substring*, and *suffix* of S, respectively. They are called a *proper* prefix, *proper* substring, and *proper* suffix of S if $x \neq S$, $y \neq S$, and $z \neq S$, respectively. If a non-empty string x is both a proper prefix and a proper suffix of S, then x is called a *border* of

S. For any $0 \leq i \leq |S|-1$, $S[i]$ denotes the i-th character of S. For any $0 \leq i \leq j \leq |S|-1$, $S[i..j]$ denotes the substring of S starting at position i and ending at position j, i.e., $S[i..j] = S[i]S[i+1]\cdots S[j]$. For convenience, $S[i..j] = \varepsilon$ for any $i > j$. A string S is called a *palindrome* if $S[i] = S[|S|-i-1]$ for every $0 \leq i \leq |S|-1$. Note that the empty string is a palindrome. A substring $S[i..j]$ of S is said to be a *palindromic substring* of S if $S[i..j]$ is a palindrome. The *center* of a palindromic substring $S[i..j]$ of S is $\frac{i+j}{2}$. A palindromic substring $S[i..j]$ of S is *maximal* w.r.t. the center position $\frac{i+j}{2}$ if $i = 0$, $j = |S|-1$, or $S[i-1..j+1]$ is not a palindrome. We denote by $lpp(S)$ (resp. $lps(S)$) the longest palindromic prefix (resp. suffix) of S. We denote by $\mathsf{DPal}(S)$ the set of all distinct palindromes in S. It is known that $|\mathsf{DPal}(S)| \leq |S|+1$ [7]. For any non-empty strings S and w, w is said to be *unique* in S if w occurs in S exactly once. Also, w is said to be *repeating* in S if w occurs in S at least twice. For convenience, we define that the empty string ε is repeating in any string. In what follows, we consider an arbitrary fixed string S of length $n > 0$.

2.2. Eertrees (palindromic trees)

The *eertree* of S denoted by $\mathsf{eertree}(S)$ is a tree-like data structure that enables us to efficiently access each of the distinct palindromes in S [4]. The $\mathsf{eertree}(S)$ consists of m *ordinary nodes* and two auxiliary nodes, denoted $\mathtt{0}$*-node* and $-\mathtt{1}$*-node*, where $m = |\mathsf{DPal}(S)| - 1$. Each ordinary node corresponds to each element of $\mathsf{DPal}(S) \setminus \{\varepsilon\}$. For each ordinary node v, we denote by $pal(v)$ the palindrome which corresponds to v, and by $len(v)$ its length. Conversely, for each non-empty palindromic substring p of S, we denote by $node(p)$ the node which corresponds to the palindrome p. Namely, $node(pal(v)) = v$ for each ordinary node v. For convenience, we define $pal(\mathtt{0}\text{-node}) = pal(-\mathtt{1}\text{-node}) = \varepsilon$, $len(\mathtt{0}\text{-node}) = 0$, and $len(-\mathtt{1}\text{-node}) = -1$. For any nodes u, v in $\mathsf{eertree}(S)$, there is an edge (u, v) if and only if $len(u) + 2 = len(v)$ and $pal(u) = pal(v)[1..len(v)-2]$. Each edge (u, v) is labeled by a character $pal(v)[0]$. Also, each node v in $\mathsf{eertree}(S)$ has a *suffix link* denoted by $slink(v)$. For each node v in $\mathsf{eertree}(S)$ with $len(v) \geq 2$, $slink(v)$ points to the node corresponding to the longest palindromic proper suffix of $pal(v)$. For each node v in $\mathsf{eertree}(S)$ with $len(v) = 1$, $slink(v)$ points to the $\mathtt{0}$-node. Also, $slink(\mathtt{0}\text{-node}) = -\mathtt{1}\text{-node}$ and $slink(-\mathtt{1}\text{-node}) = -\mathtt{1}\text{-node}$. For each node v in $\mathsf{eertree}(S)$, $inSL(v) = |\{u \mid slink(u) = v\}|$ denotes the number of incoming suffix links of v. See Fig. 1 for an example of $\mathsf{eertree}(S)$.

Note that each node v does not store the string $pal(v)$ explicitly. Instead, we can obtain $pal(v)$ by traversing edges backward, from v to the root, since $pal(u) = c\,pal(u')c$ for each node u with $|pal(u)| \geq 2$ where u' is the parent of u and c is the label of the edge (u', u). Each node only stores pointers to its children and a constant number of integers. Thus, the size of $\mathsf{eertree}(S)$ is linear in the number of nodes, i.e., $O(|\mathsf{DPal}(S)|)$. It is known that $\mathsf{eertree}(S)$ can be constructed in $O(n \log \sigma)$ time for any string S given in an online manner [4].

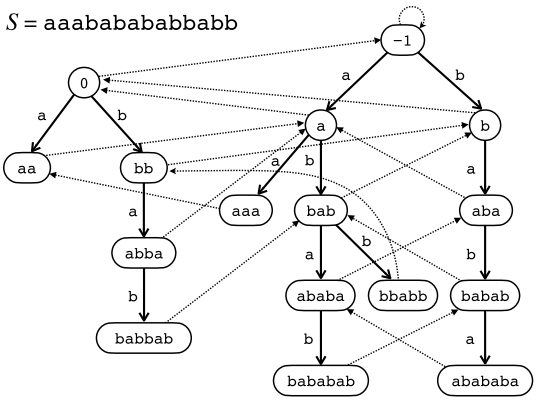

Fig. 1. The eertree of $S = \mathtt{aaababababbabb}$. The solid and broken arrows represent edges and suffix links, respectively. Note that $pal(v)$ is written inside each node v in this figure, however, it is for only explanation. Namely, each node does not explicitly store the corresponding string.

2.3. Sliding window

We formalize sliding windows over string S. For each time $t = 0, 1, \ldots$, we consider the substring $S[i_t..j_t]$ called *the window at time t*. The windows must satisfy the following conditions: (1) $i_0 = j_0 = 0$ for the initial window at time 0; and (2) $0 \leq i_t \leq j_t \leq n-1$ and either $(i_t, j_t) = (i_{t-1}+1, j_{t-1})$ or $(i_t, j_t) = (i_{t-1}, j_{t-1}+1)$ for every time $t > 0$. In other words, the second condition means that we can either *delete* the leftmost character from the current window, or *append* a character to the right end of the current window at each time.

Given a sequence of windows (or equivalently, a sequence of delete / append operations), the aim of our sliding window model is processing the windows in space proportional to the size of each window. This paper mainly deals with the problem of maintaining eertrees with respect to a sequence of windows over a given string S.

3. Combinatorial properties on palindromes for a sliding window

In this section, we show some combinatorial properties on palindromes for a sliding window, which is helpful for designing efficient algorithms to maintain eertrees for a sliding window. Since the nodes of the eertree of a string represent all distinct palindromes in the string, we obtain the next lemma.

Lemma 1. *There is a node ℓ in $\mathsf{eertree}(S[i-1..j-1])$ to be removed when the leftmost character $S[i-1]$ is deleted from $S[i-1..j-1]$ if and only if (A) $pal(\ell)$ is unique in $S[i-1..j-1]$ and (B) $pal(\ell) = lpp(S[i-1..j-1])$. Moreover when this holds, (C) ℓ is a leaf node.*

Proof. ($\Rightarrow$) (A) Since ℓ is removed, $pal(\ell)$ does not occur in $S[i..j-1]$. Thus, $pal(\ell)$ occurs in $S[i-1..j-1]$ only as a prefix, i.e., $pal(\ell)$ is unique in $S[i-1..j-1]$. (B) Assume that $pal(\ell)$ is shorter than $lpp(S[i-1..j-1])$. Then, $pal(\ell)$ is a proper prefix of $lpp(S[i-1..j-1])$. Also, $pal(\ell)$ is a proper suffix of $lpp(S[i-1..j-1])$ since $lpp(S[i-1..j-1])$

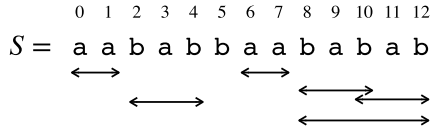

Fig. 2. For string $S = \texttt{aababbaababab}$, its palindromic substring aa is border-maximal in S. On the other hand, bab is not border-maximal in S since there is a palindromic substring $S[8..12] = \texttt{babab}$ of S which contains bab as a proper suffix.

is a palindrome. This contradicts that $pal(\ell)$ is unique in $S[i-1..j-1]$. Thus, $pal(\ell) = lpp(S[i-1..j-1])$. (C) If we assume that ℓ has a child, then $pal(\ell)$ has an occurrence in $S[i-1..j-1]$ that is not a prefix of $S[i-1..j-1]$, a contradiction.
($\Leftarrow$) Since $pal(\ell)$ is a palindromic prefix of $S[i-1..j-1]$ and unique in $S[i-1..j-1]$, $pal(\ell)$ does not occur in $S[i..j-1]$. Thus, ℓ is removed when $S[i-1]$ is deleted. □

Namely, when the leftmost character of the window is deleted, at most one leaf will be removed from the eertree. For example, in Fig. 1, the leaf corresponding to aaa will be removed when the leftmost character of S is deleted, since aaa is the longest palindromic prefix of S and it is unique in S. Also, in order to detect such a leaf, we need to compute the longest palindromic prefix of each window and to determine its uniqueness. In the following, we show some combinatorial properties on unique palindromes and the longest palindromic prefix for a sliding window.

Unique palindromes for a sliding window. A palindromic substring w of string S is said to be *border-maximal* in S if there is no palindromic substring of S, which contains w as a proper suffix. See Fig. 2 for examples. If a palindrome w is not border-maximal in S, then w is a border of another palindrome w', i.e., w is not unique in S. In other words, any unique palindromic substring must be border-maximal.

Longest prefix palindrome for a sliding window. Next, we consider the longest palindromic prefixes for sliding windows.

Lemma 2. *Let w be the longest palindromic prefix of the window $S[i_t..j_t]$ at time t. There exists time $t' \leq t$ which satisfies one of the followings:*

1. *the longest palindromic suffix of $S[i_{t'}..j_{t'}]$ is w, or*
2. *the longest palindromic suffix of $lpp(S[i_{t'}..j_{t'}])$ is w.*

(See also Fig. 3 for concrete examples.)

Proof. Let $w = S[s..e]$. If $w = S[i_t..j_t]$, then w is also the longest palindromic suffix of the window. Namely, w satisfies the first condition of the lemma for $t' = t$. Otherwise, for the sake of contradiction, we assume the contrary. Namely, for every time $t' \leq t$, neither the longest palindromic suffix of $S[i_{t'}..j_{t'}]$ nor the longest palindromic suffix of $lpp(S[i_{t'}..j_{t'}])$ is not equal to $S[s..e]$. Consider a window in the past such that its ending position is e. Since the longest palindromic suffix of the window is not $S[s..e]$, there is another palindromic suffix ending at e,

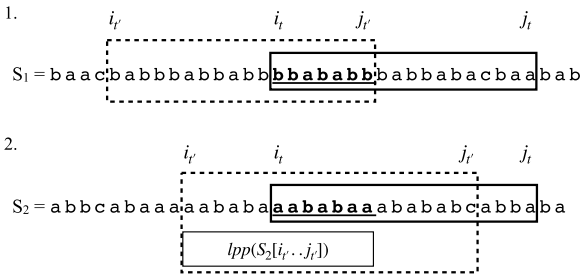

Fig. 3. The upper one shows a string S_1 and windows $S_1[i_t..j_t]$ and $S_1[i_{t'}..j_{t'}]$ with $t' \leq t$. The longest palindromic prefix bbababb of $S_1[i_t..j_t]$ is equal to the longest palindromic suffix of $S_1[i_{t'}..j_{t'}]$ (i.e., the first case of Lemma 2). The lower one shows a string S_2 and windows $S_2[i_t..j_t]$ and $S_2[i_{t'}..j_{t'}]$ with $t' \leq t$. The longest palindromic prefix aababaa of $S_2[i_t..j_t]$ is equal to the longest palindromic suffix of $lpp(S_2[i_{t'}..j_{t'}])$ (i.e., the second case of Lemma 2).

which is longer than $S[s..e]$. Now let $v = S[s'..e]$ be the shortest palindrome which is ending at e and is longer than $w = S[s..e]$. Namely, w is the longest palindromic suffix of v. Next, consider a window $S[i_{\tilde{t}}..j_{\tilde{t}}]$ at time $\tilde{t} \leq t$ with its starting position $i_{\tilde{t}} = s'$. If v is the longest palindromic prefix of the window $S[i_{\tilde{t}}..j_{\tilde{t}}]$, then w becomes the longest palindromic suffix of $v = lpp(S[i_{\tilde{t}}..j_{\tilde{t}}])$, however, it contradicts our assumption. Thus, there is another palindromic prefix starting at s', which is longer than v. Now let $u = S[s'..e']$ be the longest palindromic substring of the window $S[i_{\tilde{t}}..j_{\tilde{t}}]$.

Further let c_w, c_v, and c_u be respectively the center of w, v, and u. From the assumptions, $c_v < c_w$ and $c_v < c_u$ hold. Next, we consider three sub-cases (see also Fig. 4):

(a) If $c_u < c_w$, then the palindrome u' ending at e whose center equals c_u, is longer than w and is shorter than v. This contradicts that w is the longest palindromic suffix of v.
(b) If $c_u = c_w$, then $|v| = e - s' + 1 = e' - s + 1$. Thus, $S[s..e'] = S[s'..e] = v$ since $S[s'..e']$ is a palindrome. This contradicts that w is the longest palindromic prefix of the current window $S[i_t..j_t] = S[s..j_t]$.
(c) If $c_w < c_u$, then the palindrome u'' starting at s whose center equals c_u, is longer than w. This again contradicts that w is the longest palindromic prefix of the current window $S[i_t..j_t] = S[s..j_t]$.

Therefore, we have proved the lemma. □

4. Eertree for a sliding window

In this section, we show how to update a given eertree when we shift the sliding window to the right by one character. Sliding a given window consists of two operations: *deleting* the leftmost character and *appending* a character to the right end. Namely, when the eertree of $S[i-1..j-1]$ is given, we first compute the eertree of $S[i..j-1]$ (deleting the leftmost character), and then, compute the eertree of $S[i..j]$ (appending a character). To update the eertree when a character is appended, we can apply Rubinchik and Shur's algorithm [4] which constructs the eertree of a given string in an online manner. In this section, we pro-

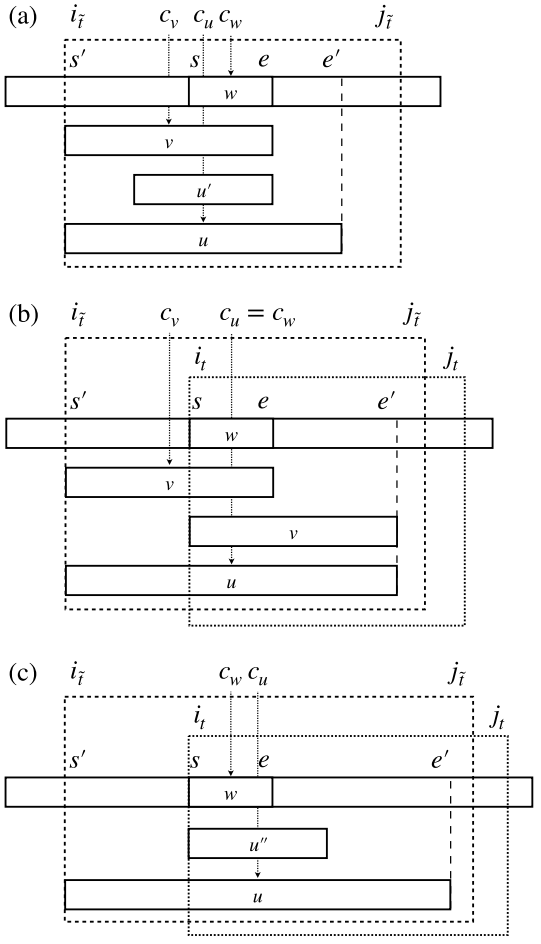

Fig. 4. Illustration for contradictions in the proof of Lemma 2.

pose new additional data structures and algorithms which update the eertree when the leftmost character is deleted.

We emphasize that our algorithms work for any valid[1] sequence of windows of arbitrary lengths. However, for simplicity, we consider the case where a fix-sized window of length d shifts to the right one-by-one throughout this section.

4.1. Auxiliary data structures for detecting the node to be deleted

We introduce auxiliary data structures for computing the longest palindromic prefixes and for determining the uniqueness of palindromes.

For computing the longest palindromic prefix. Let $prefPal[0..d-1]$ be a cyclic array of size d such that $prefPal[i_t \bmod d]$ stores the node which corresponds to the longest palindromic prefix of the window $S[i_t..j_t]$ at each time t. Namely, for every time t, $prefPal[i_t \bmod d] = node(lpp(S[i_t..j_t]))$ holds.

[1] Cf., the definition of sliding windows in Section 2.3.

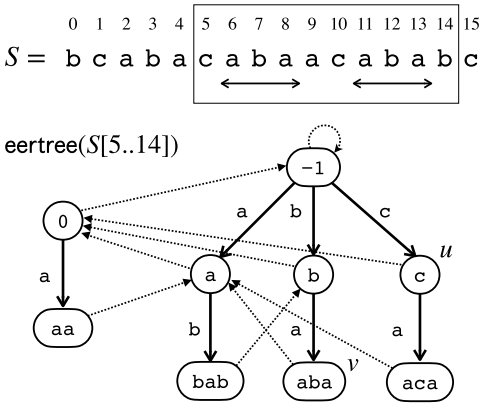

Fig. 5. Examples for $BegPair_{i,j}(v)$. For string $S = \texttt{bcabacabaacababc}$ and window $[5, 14]$, the eertree($S[5..14]$) is depicted. Consider node v in eertree($S[5..14]$) with $pal(v) = \texttt{aba}$. The rightmost and the second rightmost occurrences of aba in the window $S[5..14]$ are 11 and 6. Namely, $rm_{5,14}(v) = 11$ and $srm_{5,14}(v) = 6$. Further, $inSL(v) = 0$, and thus, $BegPair_{5,14}(v) = (11, 6)$. Also, for node u in eertree($S[5..14]$) with $pal(u) = \texttt{c}$, $BegPair_{5,14}(u) = (10, 5)$ since $rm_{5,14}(u) = 10$, $srm_{5,14}(u) = 5$, and $inSL(u) = 0$. When the leftmost character $S[5] = \texttt{c}$ is deleted from the window $S[5..14]$, $srm_{5,14}(u)$ changes to -1. However, $BegPair_{6,14}(u) = BegPair_{5,14}(u) = (10, 5)$ is allowed since $BegPair_{6,14}(u).second = 5 < 6$ is a valid value for our invariant. Namely, we do not have to update $BegPair_{6,14}(u)$ explicitly when deleting the leftmost character $S[5]$ from $S[5..14]$.

For determining uniqueness of a palindrome. For each ordinary node v in eertree($S[i..j]$), let $rm_{i,j}(v)$ be the starting position of the rightmost occurrence of $pal(v)$ in $S[i..j]$. Further let $srm_{i,j}(v)$ be the starting position of the second rightmost occurrence of $pal(v)$ in $S[i..j]$ if such a position exists, and otherwise, $srm_{i,j}(v) = -1$. Throughout the computation of the eertree for a sliding window, for each node v of eertree($S[i..j]$) we keep the following invariant $BegPair_{i,j}(v)$ which consists of two fields *first* and *second* such that: $BegPair_{i,j}(v).first = rm_{i,j}(v)$ and $BegPair_{i,j}(v).second = srm_{i,j}(v)$ if $inSL(v) = 0$, and let their values be -1 or some occurrence of $pal(v)$ in $S[0..j]$ otherwise. Namely, $BegPair_{i,j}(v)$ stores the rightmost and second rightmost occurrences of $pal(v)$ in $S[i..j]$ when $inSL(v) = 0$, if such occurrences exist. Otherwise, it temporarily stores some pair of integers, however, it will never be referred to in our algorithms. In other words, we employ a kind of lazy maintenance of the rightmost and second rightmost occurrences of $pal(v)$ in $S[i..j]$ that suffices for our purpose. See Fig. 5 for an example of $BegPair_{i,j}(v)$.

The next lemma states that given a node v, we can determine if $pal(v)$ is unique or not by checking the incoming suffix links of v and $BegPair_{i,j}(v)$.

Lemma 3. *Let v be any node in* eertree($S[i..j]$). *Then, $pal(v)$ is unique in $S[i..j]$ if and only if $inSL(v) = 0$ and $BegPair_{i,j}(v).second < i$.*

Proof. ($\Rightarrow$) We show the contraposition. There are two cases: (1) If $inSL(v) \neq 0$, then there is a palindromic substring P of $S[i..j]$ with $lps(P) = pal(v)$. Namely, $pal(v)$ is not border-maximal in $S[i..j]$, and thus, $pal(v)$ is not

Algorithm 1 *update_bp*(v, x).

Require: Node v, and a starting position x of $pal(v)$.
Ensure: Update $v.bp$ appropriately with respect to the position x.
1: **if** $x > v.bp.first$ **then**
2: $v.bp.second \leftarrow v.bp.first$
3: $v.bp.first \leftarrow x$
4: **else if** $x > v.bp.second$ **then**
5: $v.bp.second \leftarrow x$
6: **end if**

Algorithm 2 Update *BegPair* and *prefPal* when the first leftmost character is deleted.

Require: $lpsuf = node(lps(S[i-1..j-1]))$, and $v.bp = BegPair_{i-1,j-1}(v)$ for each node v in eertree$(S[i-1..j-1])$.
Ensure: $lpsuf = node(lps(S[i..j-1]))$, and $v.bp = BegPair_{i,j-1}(v)$ for each node v in eertree$(S[i..j-1])$.
1: $lppref \leftarrow prefPal[i-1]$
2: **if** $lppref = lpsuf$ **then**
3: $lpsuf \leftarrow slink(lpsuf)$
4: **end if**
5: $q \leftarrow slink(lppref)$
6: $inSL(q) \leftarrow inSL(q) - 1$
7: $x \leftarrow i - 1 + len(lppref) - len(q)$
8: $update\_bp(q, x)$
9: **if** $len(q) > len(prefPal[x])$ **then**
10: $prefPal[x] = q$
11: **end if**
12: **if** $inSL(lppref) = 0$ and $lppref.bp.second < i - 1$ **then**
13: Remove node $lppref$ from the eertree
14: **end if**

unique in $S[i..j]$. (2) If $inSL(v) = 0$ and $BegPair_{i,j}(v).second = srm_{i,j}(v) \geq i$, then $pal(v)$ occurs at least twice in $S[i..j]$ at positions $BegPair_{i,j}(v).second$ and $BegPair_{i,j}(v).first$.
($\Leftarrow$) For the sake of contradiction, we assume that $pal(v)$ is not unique in $S[i..j]$. Then, by the definition of srm, $i \leq srm_{i,j}(v) \leq j$. However, since $inSL(v) = 0$, $srm_{i,j}(v) = BegPair_{i,j}(v).second < i$, a contradiction. □

Next, we introduce our algorithms to maintain *prefPal* and *BegPair* for a sliding window, which utilizes combinatorial properties shown in Section 3.

4.2. Maintaining the auxiliary data structures

First, in Algorithm 1, we show subroutine *update_bp* which updates the member variable $v.bp$ of a given node v where $v.bp$ must be kept equal to $BegPair_{i_t,j_t}(v)$ at each time t. It will be called in the algorithms that we show later.

Next, we show our algorithms for updating data structures when we slide the given window. When the leftmost character $S[i-1]$ is deleted from $S[i-1..j-1]$, our data structures are updated by Algorithm 2. Also, when a character $S[j]$ is appended to $S[i..j-1]$, our data structures are updated by Algorithm 3.

Time complexities. Clearly, Algorithm 1 runs in constant time. In Algorithm 2, all lines except for Line 13 can be processed in constant time. Thus, the total running time of Algorithm 2 is dominated by Line 13, i.e., $O(\log \sigma')$. In Algorithm 3, the first four lines can be processed in amortized $O(\log \sigma')$ time by using the online construction algorithm [4]. Also, the remaining lines can be processed in constant time, and thus, the total running time of Algorithm 3 is amortized $O(\log \sigma')$.

Algorithm 3 Update *BegPair* and *prefPal* when a character is appended.

Require: $lpsuf = node(lps(S[i..j-1]))$, $S[j]$, and $v.bp = BegPair_{i,j-1}(v)$ for each node v in eertree$(S[i..j-1])$.
Ensure: $lpsuf = node(lps(S[i..j]))$, and $v.bp = BegPair_{i,j}(v)$ for each node v in eertree$(S[i..j])$.
1: $lpsuf \leftarrow node(lps(S[i..j]))$
2: **if** $lpsuf$ is not in eertree$(S[i..j-1])$ **then**
3: Add new node $lpsuf$ to the eertree
4: **end if**
5: $y \leftarrow j - len(lpsuf) + 1$
6: $update\_bp(lpsuf, y)$
7: $prefPal[y] \leftarrow lpsuf$

Correctness. First, it is clear that Algorithm 1 runs correctly. Next, let us consider the correctness of Algorithm 2. Let us first consider a special case when the window $S[i-1..j-1]$ itself is a palindrome. Then, we need to update $lpsuf$, which will be used in Algorithm 3. Lines 2–3 of Algorithm 2 capture such a case. Next, we show that *BegPair* for all nodes are updated correctly. By the invariant of *BegPair*, it suffices to update $v.bp$ for every node v where $inSL(v) = 0$, i.e., $pal(v)$ is border-maximal. Let q be the node corresponding to the longest palindromic suffix of $lppref = lpp(S[i-1..j-1])$. Then, it suffices to update $q.bp$ since the node q is the only candidate for a node whose corresponding palindrome to be border-maximal *just* in this step. Thus, we update only $q.bp$ in Lines 5–8, if it is needed. Further, we show that *prefPal* is also updated correctly. By Lemma 2, the longest palindromic prefix of a window must be the longest palindromic suffix of either some window or the longest palindromic prefix of some window. The palindrome $pal(q)$ is the only one that is to be such a palindrome *just* in this step. Thus, $prefPal[x]$ is the only candidate which may be updated in this step where x is the starting position of the occurrence of $pal(q)$ that is the longest palindromic suffix of $lpp(S[i-1, j-1])$. Therefore, it suffices to update $prefPal[x]$ and update it if necessary (Lines 9–11). Line 12 determines the uniqueness of $lpp(S[i-1..j-1])$ correctly by using Lemma 3, and if it is unique, then the corresponding node $lppref$ is removed (in Line 13).

Finally, consider the correctness of Algorithm 3. When a character is appended, we first check the new longest palindromic suffix, and create a new node corresponding to the palindrome if necessary. These procedures in Lines 1–4 are correctly performed by running the online construction algorithm [4]. Let y be the starting position of the longest palindromic suffix of the window $S[i..j]$. The palindrome $lps(S[i..j])$ is the only candidate for a palindrome to be border-maximal *just* in this step, and thus, it suffices to update $lpsuf.bp$ in this step. Also, $lpsuf$ is the only candidate for the node that we need to newly store into *prefPal* in this step. At this moment, $lpsuf$ is clearly the longest palindrome starting at position y. Thus, we set $prefPal[y] \leftarrow lpsuf$ (Line 7).

To summarize this section, we obtain the following theorem.

Theorem 1. *We can maintain eertrees for a sliding window in a total of $O(n \log \sigma')$ time using $O(d') + d$ space where $d' \leq d$ is the maximum number of distinct palindromes in all windows.*

Proof. When a character is appended to the right end of the window, we update the eertree itself by applying the online algorithm [4], and update our auxiliary data structures by using Algorithm 3. When the leftmost character is deleted from the window, we update the eertree and the auxiliary data structures by using Algorithm 2. The total running time is $O(n \log \sigma')$. The space usage is $O(d')$ words for the original eertree and the auxiliary member variable $v.bp$ for each node v of the eertree, plus d words for the array $prefPal[0..d-1]$. □

By applying a subtle modification to the above algorithm, we obtain another variant of the algorithm (Theorem 2 below) which is faster than Theorem 1 when $d'\sigma < n \log \sigma'$, but using additional $(d'+1)\sigma$ space.

Theorem 2. *We can maintain eertrees for a sliding window in a total of $O(n + d'\sigma)$ time using $(d'+1)\sigma + O(d') + d \in O(d\sigma)$ space.*

Proof. In the original eertrees, each node stores a binary search tree to maintain branches dynamically. Instead, we use an array of integers of size σ, which allows us to add, delete, and search for a node pointer (i.e., edge) labeled by a given character in constant time. Thus, the $\log \sigma'$ factor in our time complexity can be removed. On the other hand, we need $\sigma + O(1)$ space to represent each node object, and $\Theta(\sigma)$ time to initialize it. If we naively initialize such a node object when adding a new node, the total time complexity increases to $O(n\sigma)$. However, we can *reuse* node objects that had been removed when deleting a character since such removed nodes, and new nodes to be added are leaves, i.e., they do not have any child (Lemma 1). Thus, by reusing node objects, we do not need to initialize an array of size σ when adding a new leaf node. The total number of node objects to initialize is $d'+1$, and it costs $O(d'\sigma)$ total time to initialize them. □

5. Applications of eertrees for a sliding window

In this section, we apply our sliding-window eertree algorithm of Section 4 to computing minimal unique palindromic substrings and minimal absent palindromic words for a sliding window.

5.1. Computing minimal unique palindromic substrings for a sliding window

A substring $S[i..j]$ of S is called a *minimal unique palindromic substring (MUPS)* of S if and only if $S[i..j]$ is a palindrome, $S[i..j]$ is unique in S, and $S[i+1..j-1]$ is repeating in S. We denote $\mathsf{MUPS}(S)$ the set of intervals corresponding to MUPSs of S, i.e., $\mathsf{MUPS}(S) = \{[i, j] \mid S[i..j] \text{ is a MUPS of } S\}$. For example, palindromic substring $S[9..13] = \texttt{bbabb}$ of string $S = \texttt{aaababababbabb}$ is a MUPS of S since $S[9..13] = \texttt{bbabb}$ is unique in S and $S[10..12] = \texttt{bab}$ is repeating in S.

Now, we show Lemma 4 which states a relationship between eertrees and MUPSs. Then, in Lemma 5, we show that all MUPS can be computed using eertrees in an offline manner.

Lemma 4. *A string w is a MUPS of S if and only if there is a node v in $\mathsf{eertree}(S)$ such that $pal(v) = w$, $pal(v)$ is unique in S and $pal(u)$ is repeating in S, where u is the parent of v.*

Proof. ($\Rightarrow$) Since w is a MUPS of S, it is clear that there is a node v such that $pal(v) = w$ and it is unique in S. Also, since $pal(v) = w \neq \varepsilon$, v has the parent u, which represents the string $w[1..|w|-2]$. By the definition of MUPS, $pal(u) = w[1..|w|-2]$ is repeating in S.
($\Leftarrow$) Since the palindrome $pal(v) = w$ is unique in S and $pal(u) = w[1..|w|-2]$ is repeating in S, w is a MUPS of S. □

Lemma 5. *Given $\mathsf{eertree}(S)$, we can compute $\mathsf{MUPS}(S)$ in $O(|\mathsf{DPal}(S)|)$ time.*

Proof. Given a node v, we can detect whether $pal(v)$ is a MUPS or not in constant time by Lemma 4. Also, the starting position of a palindrome $pal(v)$, which is unique in S is stored in $v.bp.first$. Therefore, we can compute $\mathsf{MUPS}(S)$ by a single traversal on $\mathsf{eertree}(S)$. □

Moreover, we can efficiently maintain MUPSs for a sliding window.

Lemma 6. *We can maintain the set of MUPSs for a sliding window in a total of $O(n \log \sigma')$ time using $O(d)$ space.*

Proof. In addition to the eertree data structure described in Section 4, we add 1-bit information ismups into each node. This bit ismups is set to `1` if the node corresponds to a MUPS and to `0` otherwise. We first consider to delete the leftmost character $S[i-1]$ from $S[i-1..j-1]$. In this case, only prefixes of $S[i-1..j-1]$ are those whose number of occurrences in the sliding window change. We check the nodes corresponding to the longest and the second longest palindromic prefixes, and update ismups of them accordingly. We do not need to care about other palindromic prefixes since they must be repeating in $S[i..j-1]$. Symmetrically, we can easily maintain ismups in the case when appending a character $S[j]$ to $S[i..j-1]$. □

5.2. Computing minimal absent palindromic words for a sliding window

A string w is called a *minimal absent palindromic word (MAPW)* of string S if and only if w is a palindrome, w does not occur in S, and $w[1..|w|-2]$ occurs in S. For example, palindrome $w = \texttt{aabbaa}$ is a MAPW of string $S = \texttt{aaababababbabb}$ since w does not occur in S and $w[1..|w|-2] = \texttt{abba}$ occurs in S at position 8. For a relation between MAPWs and eertrees, the following lemmas hold.

Lemma 7. *For any non-empty string $w \in \Sigma^*$, w is a MAPW of a string S if and only if there is a node u in* eertree(S) *such that $pal(u) = w[1..|w|-2]$, $len(u) = |w|-2$, and u does not have an edge labeled by $w[0]$.*

Proof. ($\Rightarrow$) Since w is a MAPW, $w[1..|w|-2]$ is a palindromic substring of S, and thus, there is a node u with $pal(u) = w[1..|w|-2]$ and $len(u) = |w|-2$. Also, since the palindrome w does not occur in S, u does not have an edge labeled by $w[0]$.
($\Leftarrow$) Since u is a node in eertree(S), the string $pal(u) = w[1..|w|-2]$ occurs in S. Also, since u does not have an edge labeled by $w[0]$, the string w does not occur in S. Thus, w is a MAPW of S. □

Lemma 8. *Let* MAPW(S) *be the set of MAPWs of S. Then* $|\mathsf{MAPW}(S)| = 2\sigma + (|\mathsf{DPal}(S)| - 1)(\sigma - 1)$. *Also, given* eertree$(S)$, MAPW$(S)$ *can be computed in $O(|\mathsf{DPal}(S)|\sigma)$ time.*

Proof. We prove $|\mathsf{MAPW}(S)| = 2\sigma + (|\mathsf{DPal}(S)| - 1)(\sigma - 1)$ by induction. If $S = \varepsilon$, then $\mathsf{DPal}(S) = \{\varepsilon\}$ and $\mathsf{MAPW}(S)$ consists of a and aa for each character $a \in \Sigma$. Thus, $|\mathsf{MAPW}(\varepsilon)| = 2\sigma = 2\sigma + (|\mathsf{DPal}(\varepsilon)| - 1)(\sigma - 1)$ holds. We assume that $|\mathsf{MAPW}(S)| = 2\sigma + (|\mathsf{DPal}(S)| - 1)(\sigma - 1)$ holds for any string S of length $k \geq 0$. Then consider any string S' of length $k+1$, and let $S' = Sc$ where $c \in \Sigma$. If $\mathsf{DPal}(S) = \mathsf{DPal}(Sc)$, then it is clear that $\mathsf{MAPW}(Sc) = \mathsf{MAPW}(S)$, and hence, $|\mathsf{MAPW}(Sc)| = |\mathsf{MAPW}(S)| = 2\sigma + (|\mathsf{DPal}(S)| - 1)(\sigma - 1) = 2\sigma + (|\mathsf{DPal}(Sc)| - 1)(\sigma - 1)$. Otherwise, it is known that $\mathsf{DPal}(S) \setminus \mathsf{DPal}(Sc) = \emptyset$ and $\mathsf{DPal}(Sc) \setminus \mathsf{DPal}(S) = \{lps(Sc)\}$ [7]. Let $P = lps(Sc)$ be the only palindrome in $\mathsf{DPal}(Sc) \setminus \mathsf{DPal}(S)$. Since P is absent from S and $P[1..|P|-2]$ occurs in S (at least as a suffix of S), P is in $\mathsf{MAPW}(S)$. Also, P occurs in Sc and aPa is absent from Sc for each character a, and hence, $|\mathsf{MAPW}(Sc)| = |\mathsf{MAPW}(S)| + (\sigma - 1)$ holds. Therefore, $|\mathsf{MAPW}(Sc)| = |\mathsf{MAPW}(S)| + (\sigma - 1) = 2\sigma + |\mathsf{DPal}(S)|(\sigma - 1) = 2\sigma + (|\mathsf{DPal}(Sc)| - 1)(\sigma - 1)$.

Next, we prove the time complexity for computing MAPWs with eertree(S). By Lemma 7, an ordinary node v does not have an edge labeled by c if and only if $c\,pal(v)c \in \mathsf{MAPW}(S)$. Also, a MAPW of length 1 is an absent character. Thus, it is easy to see that the information of all MAPWs can be computed by traversing eertree(S) only once. □

Also, we can maintain MAPWs for a sliding window by applying Theorem 2.

Corollary 1. *We can maintain the set of MAPWs for a sliding window in a total of $O(n + d\sigma)$ time using $O(d\sigma)$ space.*

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to thank the anonymous referees for their helpful comments on an earlier version of this paper. This work was supported by JSPS KAKENHI Grant Numbers JP20J11983 (TM), JP18K18002 (YN), JP17H01697 (SI), JP20H04141 (HB), JP18H04098 (MT), and by JST PRESTO Grant Number JPMJPR1922 (SI).

References

[1] G.K. Manacher, A new linear-time "on-line" algorithm for finding the smallest initial palindrome of a string, J. ACM 22 (3) (1975) 346–351.
[2] R. Groult, É. Prieur, G. Richomme, Counting distinct palindromes in a word in linear time, Inf. Process. Lett. 110 (20) (2010) 908–912.
[3] D. Kosolobov, M. Rubinchik, A.M. Shur, Finding distinct subpalindromes online, in: Proceedings of the Prague Stringology Conference 2013, 2013, pp. 63–69.
[4] M. Rubinchik, A.M. Shur, EERTREE: an efficient data structure for processing palindromes in strings, Eur. J. Comb. 68 (2018) 249–265.
[5] G. Fici, T. Gagie, J. Kärkkäinen, D. Kempa, A subquadratic algorithm for minimum palindromic factorization, J. Discret. Algorithms 28 (2014) 41–48.
[6] H. Bannai, T. Gagie, S. Inenaga, J. Kärkkäinen, D. Kempa, M. Piatkowski, S. Sugimoto, Diverse palindromic factorization is NP-complete, Int. J. Found. Comput. Sci. 29 (2) (2018) 143–164.
[7] X. Droubay, J. Justin, G. Pirillo, Episturmian words and some constructions of de Luca and Rauzy, Theor. Comput. Sci. 255 (1–2) (2001) 539–553.
[8] E. Ukkonen, On-line construction of suffix trees, Algorithmica 14 (3) (1995) 249–260.
[9] H. Inoue, Y. Nakashima, T. Mieno, S. Inenaga, H. Bannai, M. Takeda, Algorithms and combinatorial properties on shortest unique palindromic substrings, J. Discret. Algorithms 52–53 (2018) 122–132.
[10] K. Watanabe, Y. Nakashima, S. Inenaga, H. Bannai, M. Takeda, Fast algorithms for the shortest unique palindromic substring problem on run-length encoded strings, Theory Comput. Syst. 64 (7) (2020) 1273–1291.
[11] J. Ziv, A. Lempel, A universal algorithm for sequential data compression, IEEE Trans. Inf. Theory 23 (3) (1977) 337–343.
[12] J.G. Cleary, I.H. Witten, Data compression using adaptive coding and partial string matching, IEEE Trans. Commun. 32 (4) (1984) 396–402.
[13] E.R. Fiala, D.H. Greene, Data compression with finite windows, Commun. ACM 32 (4) (1989) 490–505.
[14] N.J. Larsson, Extended application of suffix trees to data compression, in: DCC 1996, 1996, pp. 190–199.
[15] M. Senft, Suffix tree for a sliding window: an overview, in: WDS 2005, 2005, pp. 41–46.
[16] M. Crochemore, A. Héliou, G. Kucherov, L. Mouchard, S.P. Pissis, Y. Ramusat, Absent words in a sliding window with applications, Inf. Comput. 270 (2020) 104461.
[17] T. Mieno, Y. Kuhara, T. Akagi, Y. Fujishige, Y. Nakashima, S. Inenaga, H. Bannai, M. Takeda, Minimal unique substrings and minimal absent words in a sliding window, in: SOFSEM 2020, 2020, pp. 148–160.
[18] L. Ilie, W.F. Smyth, Minimum unique substrings and maximum repeats, Fundam. Inform. 110 (1–4) (2011) 183–195.
[19] S. Yamamoto, T. Yamamoto, T. Kataoka, E. Kuramoto, O. Yano, T. Tokunaga, Unique palindromic sequences in synthetic oligonucleotides are required to induce IFN [correction of INF] and augment IFN-mediated [correction of INF] natural killer activity, J. Immunol. 148 (12) (1992) 4072–4076.
[20] E. Kuramoto, O. Yano, Y. Kimura, M. Baba, T. Makino, S. Yamamoto, T. Yamamoto, T. Kataoka, T. Tokunaga, Oligonucleotide sequences required for natural killer cell activation, Jpn. J. Cancer Res. 83 (11) (1992) 1128–1131.
[21] D. Gusfield, Algorithms on Strings, Trees, and Sequences - Computer Science and Computational Biology, Cambridge University Press, 1997.
[22] M. Giel-Pietraszuk, M. Hoffmann, S. Dolecka, J. Rychlewski, J. Barciszewski, Palindromes in proteins, J. Protein. Chem. 22 (2) (2003) 109–113.

[23] A. Glen, J. Justin, S. Widmer, L.Q. Zamboni, Palindromic richness, Eur. J. Comb. 30 (2) (2009) 510–531.
[24] M. Lothaire, Combinatorics on Words, second edition, Cambridge Mathematical Library, Cambridge University Press, 1997.
[25] M. Crochemore, F. Mignosi, A. Restivo, S. Salemi, Data compression using antidictionaries, Proc. IEEE 88 (11) (2000) 1756–1768.
[26] F. Mignosi, A. Restivo, M. Sciortino, Words and forbidden factors, Theor. Comput. Sci. 273 (1–2) (2002) 99–117.
[27] S. Chairungsee, M. Crochemore, Using minimal absent words to build phylogeny, Theor. Comput. Sci. 450 (2012) 109–116.
[28] T. Ota, H. Morita, On a universal antidictionary coding for stationary ergodic sources with finite alphabet, in: ISITA 2014, 2014, pp. 294–298.
[29] Y. Fujishige, Y. Tsujimaru, S. Inenaga, H. Bannai, M. Takeda, Computing DAWGs and minimal absent words in linear time for integer alphabets, in: MFCS 2016, 2016, pp. 38:1–38:14.